

Bài 2: Làm chủ Responsive Design với Tailwind CSS

Thời lượng ước tính: 45 phút | Tài liệu chuyên sâu

1. Tư duy Utility-First trong Tailwind

Tailwind CSS là một framework utility-first. Thay vì viết CSS thuần trong các tệp tách biệt, bạn sẽ áp dụng trực tiếp các class vào HTML.

1. Tại sao chọn Utility-First?

Cách tiếp cận này giúp xây dựng giao diện nhanh chóng. Không còn việc phải suy nghĩ tên class hay lo lắng về các style xung đột.

Tiêu đề Card

Nội dung được style bằng Tailwind.

- **Tốc độ:** Viết UI trực tiếp trong HTML.

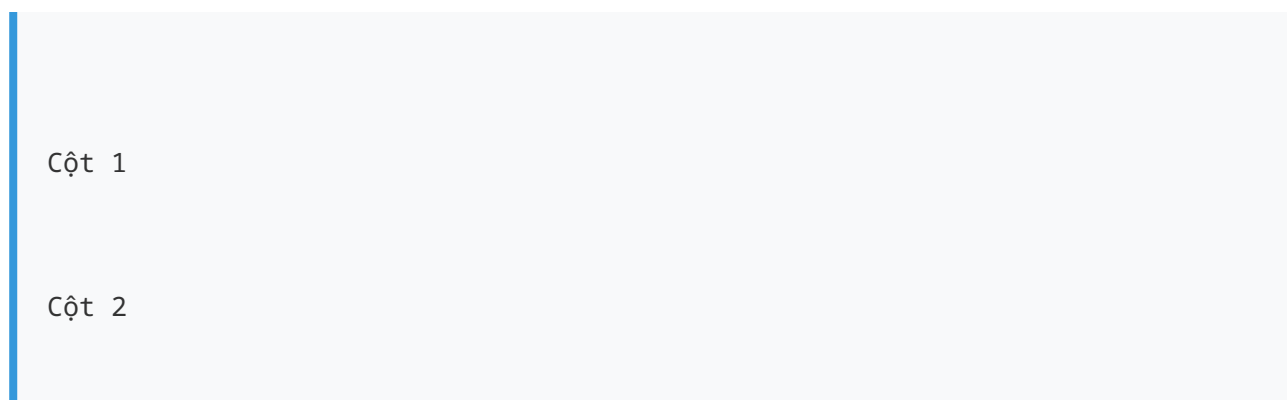
- **File size nhỏ:** Tự động loại bỏ class dư thừa (PurgeCSS).
- **Đồng nhất:** Sử dụng hệ thống design tokens có sẵn.

2. Kỹ thuật Responsive Design

Tailwind sử dụng phương pháp mobile-first. Bạn sẽ style cho màn hình nhỏ trước, sau đó dùng các breakpoint cho màn hình lớn hơn.

1. Cơ chế Breakpoints

Các breakpoint mặc định thiết kế theo kích thước thiết bị chuẩn.



- **Mặc định (Mobile):** Mọi màn hình.
- **sm: (640px):** Tablet nhỏ.
- **md: (768px):** Tablet ngang / Laptop.
- **lg: (1024px):** Desktop chuẩn.

2. Hover và Focus States

Xử lý trạng thái trực tiếp bằng các prefix utility.

Click me

Phần thực hành tổng hợp

Để củng cố kiến thức, học viên cần hoàn thành các bài tập dưới đây. Hãy ứng dụng các kiến thức đã học ở các phần trên vào việc xử lý các bài toán lập trình thực tế. Cấu trúc mã rõ ràng và có bình luận đầy đủ sẽ giúp việc bảo trì sau này dễ dàng hơn.

Bài tập 1: Triển khai kiến thức vào thực tế

Hãy phân tích yêu cầu thiết kế logic. Việc đầu tiên là định hình cấu trúc trước khi gõ những dòng mã đầu tiên. Hãy dành thời gian suy nghĩ về edge cases (các trường hợp ngoại lệ) có thể xảy ra.

```
// Vùng viết mã dành cho bài tập số 1
// TODO: Viết logic xử lý tại đây...
```

Bài tập 2: Triển khai kiến thức vào thực tế

Hãy phân tích yêu cầu thiết kế logic. Việc đầu tiên là định hình cấu trúc trước khi gõ những dòng mã đầu tiên. Hãy dành thời gian suy nghĩ về edge cases (các trường hợp ngoại lệ) có thể xảy ra.

```
// Vùng viết mã dành cho bài tập số 2  
// TODO: Viết logic xử lý tại đây...
```

Bài tập 3: Triển khai kiến thức vào thực tế

Hãy phân tích yêu cầu thiết kế logic. Việc đầu tiên là định hình cấu trúc trước khi gõ những dòng mã đầu tiên. Hãy dành thời gian suy nghĩ về edge cases (các trường hợp ngoại lệ) có thể xảy ra.

```
// Vùng viết mã dành cho bài tập số 3  
// TODO: Viết logic xử lý tại đây...
```